Chapitre 8 — Fichiers et gestion des erreurs

Prérequis : chapitres 1 à 7 · **Exercices :** `E08_Fichiers_et_erreurs`

Objectifs

- écrire, lire et compléter un fichier texte ;
- vérifier l'ouverture d'un fichier et le succès des opérations ;
- lire des valeurs numériques depuis un fichier et traiter les cas d'erreur ;
- intercepter une exception levée par la bibliothèque standard.

Sommaire

- Chapitre 8 — Fichiers et gestion des erreurs
  - Objectifs
  - Sommaire
  - 1. Les flux de fichiers
  - 2. Écrire et vérifier
  - 3. Lire un fichier ligne par ligne
  - 4. Lire des valeurs numériques
  - 5. Fermer un fichier
  - 6. Les exceptions
    - 6.1 Principe
    - 6.2 Convertir un texte en nombre : `std::stoi`
    - 6.3 Tester ou intercepter ?
  - Erreurs fréquentes
  - Synthèse

1. Les flux de fichiers

Un fichier conserve des données après la fin du programme : mesures, configuration, journal de fonctionnement. Les classes de fichiers sont déclarées dans `<fstream>` et s'utilisent comme `std::cout` et `std::cin`, avec `<<` et `>>`.

Classe	Utilisation
`std::ofstream`	Écriture
`std::ifstream`	Lecture
`std::fstream`	Lecture et écriture

2. Écrire et vérifier

```cpp
#include <fstream>
#include <iostream>

int main() {
    std::ofstream fichier{"mesures.txt"};

    if (!fichier) {                                  // ouverture a verifier
        std::cerr << "Erreur : impossible d'ouvrir le fichier.\n";
        return 1;
    }

    fichier << "Temperature : 25\n";
    fichier << "Humidite : 60\n";

    fichier.close();                                 // termine l'ecriture (§5)
    if (!fichier) {                                  // ecriture a verifier (disque
plein...)
        std::cerr << "Erreur pendant l'ecriture.\n";
        return 1;
    }

    return 0;
}
```

- Par défaut, `std::ofstream` **crée** le fichier s'il n'existe pas et **remplace** son contenu s'il existe.
- Pour **ajouter** à la fin sans effacer (cas d'un journal), on ouvre en mode `std::ios::app` : `std::ofstream journal{"journal.txt", std::ios::app};`.
- Le fichier est créé dans le **répertoire de travail** du programme, qui n'est pas forcément le dossier du fichier source.
- Une ouverture peut échouer : fichier inexistant (en lecture), chemin incorrect, droits insuffisants. On teste **toujours** le flux après l'ouverture.

Remarque : les écritures passent par un **tampon** : les données ne sont réellement écrites sur le disque que lorsque le tampon est vidé. Une erreur d'écriture peut donc n'apparaître qu'à ce moment-là. Tester le flux **après** `close()` est le contrôle le plus fiable.

3. Lire un fichier ligne par ligne

```cpp
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ifstream fichier{"mesures.txt"};

    if (!fichier) {
        std::cerr << "Erreur : impossible de lire le fichier.\n";
        return 1;
    }

    std::string ligne{};
    while (std::getline(fichier, ligne)) {   // s'arrete a la fin du fichier
        std::cout << ligne << '\n';
    }

    return 0;
}
```

Résultat, avec le fichier écrit au §2 :

```
Temperature : 25
Humidite : 60
```

4. Lire des valeurs numériques

Avec une valeur par ligne, `>>` lit directement les nombres. La lecture s'arrête à la fin du fichier **ou** à la première donnée non numérique.

```cpp
#include <fstream>
#include <iostream>
#include <vector>

int main() {
    std::ifstream fichier{"releves.txt"};

    if (!fichier) {
        std::cerr << "Erreur : fichier releves.txt introuvable.\n";
        return 1;
    }

    std::vector<double> valeurs{};
    double valeur{};

    while (fichier >> valeur) {
        valeurs.push_back(valeur);
    }

    if (!fichier.eof()) {             // arret avant la fin : donnee invalide
        std::cerr << "Attention : lecture interrompue par une donnee invalide.\n";
    }

    if (valeurs.empty()) {            // evite une division par zero
        std::cerr << "Aucune valeur exploitable.\n";
        return 1;
    }

    double somme{0.0};
    for (double v : valeurs) {
        somme += v;
    }

    std::cout << valeurs.size() << " valeur(s), moyenne : "
              << somme / static_cast<double>(valeurs.size()) << '\n';

    return 0;
}
```

Si la boucle s'arrête **sans** avoir atteint la fin du fichier (`eof()` faux), c'est qu'une donnée n'a pas pu être lue comme un nombre.

5. Fermer un fichier

Un flux est **fermé automatiquement** à la fin du bloc où il a été déclaré. `close()` permet de le fermer plus tôt, par exemple pour relire un fichier qu'on vient d'écrire :

```cpp
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ofstream sortie{"resultats.txt"};
    if (!sortie) {
        std::cerr << "Erreur : impossible de creer resultats.txt\n";
        return 1;
    }
    sortie << "Test termine\n";
    sortie.close();

    std::ifstream entree{"resultats.txt"};
    if (!entree) {
        std::cerr << "Erreur : impossible de relire resultats.txt\n";
        return 1;
    }
    std::string ligne{};
    std::getline(entree, ligne);
    std::cout << ligne << '\n';

    return 0;
}
```

Résultat :

```
Test termine
```

Les saisies au clavier se contrôlent selon le même principe : voir chapitre 3 (§4) et chapitre 4 (§6).

6. Les exceptions

6.1 Principe

Certaines fonctions de la bibliothèque standard signalent une erreur en **levant une exception** : elles interrompent le traitement en cours. Si l'exception n'est pas interceptée, le programme s'arrête aussitôt avec un message du type :

```
terminate called after throwing an instance of 'std::out_of_range'
```

Pour **intercepter** une exception, on place les instructions à risque dans un bloc `try`, suivi d'un ou plusieurs blocs `catch` :

```cpp
#include <iostream>
#include <stdexcept>
#include <vector>

int main() {
    std::vector<int> valeurs{10, 20, 30};

    try {
        std::cout << valeurs.at(5) << '\n';        // indice invalide : exception
        std::cout << "Ligne jamais executee\n";
    } catch (const std::out_of_range&) {
        std::cout << "Indice invalide\n";
    }

    std::cout << "Le programme continue\n";

    return 0;
}
```

Résultat :

```
Indice invalide
Le programme continue
```

- Dès qu'une exception est levée dans le `try`, la suite du bloc est **abandonnée** et l'exécution passe au `catch` correspondant.
- Le `catch` reçoit l'exception par **référence constante** ; son type est déclaré dans `<stdexcept>`.
- `catch (const std::exception& e)` intercepte toutes les exceptions standard ; `e.what()` renvoie un texte qui décrit l'erreur.

6.2 Convertir un texte en nombre : `std::stoi`

`std::stoi(texte)` (en-tête `<string>`) convertit un texte en `int` ; `std::stod` fait de même vers un `double`. C'est utile pour exploiter une ligne lue avec `std::getline`. Ces fonctions lèvent :

- `std::invalid_argument` si le texte ne commence pas par un nombre ;
- `std::out_of_range` si le nombre ne tient pas dans le type.

```cpp
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> textes{"42", "abc", "99999999999"};

    for (const std::string& texte : textes) {
        try {
            int valeur{std::stoi(texte)};
            std::cout << texte << " -> " << valeur << '\n';
        } catch (const std::invalid_argument&) {
            std::cout << texte << " : pas un nombre\n";
        } catch (const std::out_of_range&) {
            std::cout << texte << " : hors limites\n";
        }
    }

    return 0;
}
```

Résultat :

```
42 -> 42
abc : pas un nombre
99999999999 : hors limites
```

Attention : `std::stoi("12abc")` renvoie `12` **sans** exception : la conversion s'arrête au premier caractère non numérique.

6.3 Tester ou intercepter ?

Situation	Outil
Erreur prévisible et normale (fichier absent, saisie invalide, valeur hors limites)	Test avec `if`
Erreur signalée par une fonction de la bibliothèque (`at()`, `std::stoi`…)	`try` / `catch`

On ne remplace pas les tests par des exceptions : un `if` reste la façon la plus claire de traiter un cas attendu.

Erreurs fréquentes

Erreur	Correction
Ouverture non testée	`if (!fichier)` après l'ouverture
Fichier « introuvable » alors qu'il existe	Vérifier le répertoire de travail ou donner un chemin complet
Journal effacé à chaque exécution	Mode `std::ios::app`
Moyenne d'un fichier vide	Tester `empty()`
Écriture supposée réussie	Tester le flux après `close()`
`std::stoi` sur un texte non vérifié	`try` / `catch` (`std::invalid_argument` , `std::out_of_range`)
Exception non interceptée	Le programme s'arrête : prévoir un `catch` adapté

Synthèse

- `std::ofstream` écrit (et remplace), `std::ifstream` lit, `std::ios::app` ajoute à la fin.
- On teste le flux après l'ouverture et, pour une écriture importante, après `close()` .
- `while (std::getline(f, ligne))` lit des lignes ; `while (f >> valeur)` lit des valeurs ; `eof()` distingue la fin normale d'une donnée invalide.
- Un flux se ferme automatiquement en fin de bloc ; `close()` le ferme plus tôt.
- Une exception non interceptée arrête le programme ; `try` / `catch` permet de la traiter. `std::stoi` et `at()` en lèvent.
- Les cas d'erreur prévisibles se traitent avec `if` ; `try` / `catch` sert aux erreurs signalées par la bibliothèque.